

MiniChess AI — Demo 速查小抄

PVS 搜尋引擎 · 對 Weak / Strong / Boss 全勝(10:0) · 可編輯範圍:state.cpp、state.hpp、src/policy/*

0. 開場必講(先背這兩段)

一句話總結:「我實作一個以 **negamax** 為基礎、採用**迭代加深**的 **PVS(主變例搜尋)**引擎;核心是 **alpha-beta + PVS**,搭配**置換表**、**走法排序**、**靜止搜尋**,以及**延伸/縮減/裁剪**等選擇性深度技術,最後餵給結合**材料**、**位置**、**王安全**、**兵形**的評估函數。」

最大優化(幾乎必問):「把根節點最佳走法存入置換表。原本限時搜尋被中斷時會回報一個會輸的吃子;修正後每層都先回報真正最佳手,對 Weak 由 **40% 提升到 100%**。」

1. 整體架構(分層強化)

- 骨架:Negamax + 迭代加深(Iterative Deepening)
- 核心剪枝:Alpha-Beta → PVS
- 加速結構:置換表(TT)、走法排序、靜止搜尋
- 選擇性深度:將軍延伸、LMR、空著裁剪、期望窗口
- 評估:材料、位置表(PST)、王接近度、通路兵、車開放線、兵形、機動力

這樣講:「每一層都讓搜尋更快或更準,相同時間內看得更深。」

2. Negamax(負極大值)— Baseline

零和賽局用單一視角:每個節點取最大,並對子節點取負,一條公式處理雙方。

```
if (game_state == WIN)
    return P_MAX - ply;    // 越快將死越好
int best = M_MAX;
for (auto& a : legal_actions) {
    State* nx = next_state(a);
    int score = -eval_ctx(nx, depth-1, ...); // 取負
    if (score > best) best = score;          // 取最大
}
return best;
```

- 正分 = 對走子方有利;**score = -eval_ctx(child)** 是核心。
- 終端:可吃王回傳 **P_MAX - ply**(減 ply 讓引擎偏好更快的將死)。

Q: 為什麼用 negamax 而不分開寫 min/max?

A: 零和下一方得分=另一方失分,取負即可統一兩方,程式更簡潔、較不易出錯。

3. Alpha-Beta 剪枝 — Baseline

用 $[\alpha, \beta]$ 窗口剪掉對手不會允許的分枝。

```
for (auto& a : moves) {    // 已用 MVV-LVA 排序
    int score = -eval_ctx(nx, d-1, -beta, -alpha, ...);
    if (score > best) best = score;
    if (score > alpha) alpha = score;
    if (alpha >= beta) break; // ← beta cutoff(關鍵)
}
```

- α :我方已保證的下限; β :對手會容許的上限。
- **score $\geq \beta$** → 對手不會走到此局面 → 剪枝,結果與完整 minimax 相同。
- 最佳排序下複雜度約 $O(b^{(d/2)})$,等於相同時間深度可加倍。

Q: 為什麼剪枝不會改變結果?

A: 被剪掉的分枝對手本來就不會選(已 $\geq \beta$),所以不影響根節點的最佳值。

4. PVS 主變例搜尋 — 實際對局用

假設(排序好時)第一手最好:先算主變例,其餘手用零寬窗口快篩。

```
if (first) {          // 主變例:完整窗口
    score = -eval(nx, d-1, -beta, -alpha);
} else {              // 其餘:零寬窗口
    score = -eval(nx, d-1, -alpha-1, -alpha);
    if (score > alpha && score < beta) // 意外更優
        score = -eval(nx, d-1, -beta, -alpha); // 重搜
}
```

- **零寬窗口 $[\alpha, \alpha+1]$** :只做「是否比 PV 更好」的布林測試,快很多。
- 排序良好時重搜很少,因此比一般 alpha-beta 更快。

Q: 零寬窗口是什麼?為什麼快?

A: 把搜尋窗口縮到最小,只判斷該手是否超過 α ;不必算出精確分數,能更早 cutoff。

5. 走法排序(讓剪枝有效)

好棋先試,才會更早 cutoff。順序:置換表手 → 吃子 → 殺手 → 安靜手。

```
// 吃子:MOV-LVA;殺手:-1;安靜:-10
int as = av ? (piece_val[av]*10 - piece_val[aa])
    : (a_kill ? -1 : -10);
```

- **MOV-LVA**:吃最大子、用最小子吃(victim 大、attacker 小者優先)。
- **殺手走法**:每層保留 2 個曾造成 cutoff 的安靜手,優先嘗試。

Q: 走法排序為何重要?

A: alpha-beta 的效率完全取決於好棋是否先試;先試好棋 → 更早 cutoff → 同時間看更深。

6. 靜止搜尋(Quiescence)

葉節點只追吃子到「安靜」才評估,避免視界效應。

```
int stand_pat = evaluate(...);
if (stand_pat >= beta) return beta; // 不吃也夠好
if (stand_pat > alpha) alpha = stand_pat;
// delta 剪枝:吃了也拉不回 alpha 就跳過
if (stand_pat + capture_val[cap] + 50 < alpha) continue;
```

- **視界效應**:在深度 0 直接評估可能剛好停在被吃子前而誤判佔優。
- **stand-pat**=「不吃」的基準;**delta pruning** 跳過無望吃子。上限 MAX_QDEPTH=8。

Q: 靜止搜尋解決什麼問題?

A: 讓葉節點在戰術上穩定——不會因為剛好停在交換中間而高估或低估。

7. 置換表 與 Zobrist 雜湊

同一局面常被不同走序到達;以 100 萬筆雜湊表快取結果。

```
TTEEntry& e = g_tt[key & TT_MASK]; // 探測
if (e.key == key && e.depth >= depth) {
    if (e.flag == TT_EXACT) return e.score;
}
h ^= zob[p][orig][from]; // Zobrist 移出(走子時增量)
h ^= zob[p][moved][to]; // Zobrist 移入
```

- 每(子,格,顏色)對應一隨機 64-bit 數,盤面雜湊=各子 XOR;走子 $O(1)$ 增量更新。
- 存 key(驗證碰撞)、深度、分數、界限(精確/下界/上界)、最佳走法;深度優先替換。
- 將死分數依距根步數 ply 正規化(存入加 ply、取出減 ply)。

Q: 怎麼處理雜湊碰撞與將死分數?

A: entry 內存完整 key,取用前比對;將死分數做 ply 正規化,避免不同深度取用造成步數錯亂。

8. 選擇性深度(延伸 / 縮減 / 裁剪)

```
int ext = (ply<20 && nx->is_king_attacked()) ? 1 : 0; // 將軍延伸
if (!ext && depth>=3 && move_index>=3 && !capture) // LMR
    reduction = (depth>=6) ? 2 : 1;
if (null_score >= beta) return beta; // 空著裁剪
```

- **將軍延伸**:被將軍的著法多搜 1 層,把強制應對算清楚。

- **LMR**:排序後段、非吃子、非將軍的手先搜淺;意外 $>\alpha$ 才以完整深度重搜。
- **空著裁剪**:跳一手仍 β 就剪枝;接近將死、低子力、被將軍時跳過。
- **期望窗口**:用上一層分數 ± 50 的窄窗口,提高剪枝率。

Q: LMR / 空著裁剪有風險嗎?

A: 都是啟發式可能漏算戰術;故 LMR 限定後段安靜手且會重搜,空著裁剪在危險局面跳過,以控制風險。

9. 評估函數(為局面打分)

分數 = 我方總分 - 對方總分(走子方視角,正=有利)。

```
self_score += kp_material[piece];    // 材料
self_score += pst[piece-1][row][col]; // 位置表 PST
self_score += king_tropism(...);    // 王接近度
// + 通路兵 / 車開放線 / 兵形罰分
bonus += 3 * (self_mob - oppn_mob); // 機動力
return self_score - oppn_score + bonus;
```

- 材料:兵20 馬60 象65 車100 后200(比例符合官方 100 步判定:兵1 車5 馬3 象3 后9)。
- 位置表、王接近度、通路兵、車開放線、兵形(疊兵 $\square 15$ /孤兵 $\square 10$)、機動力(步數差 $\times 3$)。

Q: 100 步判定如何影響策略?

A: 100 步內未吃王則比子力,故長局需確保子力領先;引擎材料權重比例與官方判定一致。

10. 關鍵修復:根節點存入置換表(重點!)

問題:原本 PVS::search 每層都重新用吃子排序根節點,把「會輸的吃子」排第一;測試程式時間一到就 kill 引擎並取「最後回報的走法」,深層被中斷時就走出敗著(開局第 7 手被將死)。固定深度本來都正確,只有限時會出錯。

```
// 每層搜尋結束:把根節點最佳手寫入 TT
if (best_move != Move{} && !ctx.stop) {
    TTEntry& rt = g_tt[root_key & TT_MASK];
    rt.key = root_key;
    rt.best_move = pack_move(best_move);
    rt.depth = depth; rt.flag = TT_EXACT;
}
```

解法/結果:每層先搜、先回報真正最佳手,即使被中斷也是合理走法。**Weak 40% $\rightarrow$ 100%**,Strong、Boss 同步提升。

這樣講:「我最大的優化是把根節點最佳手存進置換表。因為比賽是限時、時間到就抓最後回報的手;舊版排序會先報一個會輸的吃子,所以會被將死。修好之後每一步都穩定,三個對手都 10:0。」

11. 成果(每步 2000ms、各 10 局、雙色輪流)

對手	修正前	修正後
Weak	40%	100%(10:0)
Strong	約 90%	100%(10:0)
Boss	約 70%	100%(10:0)

12. TA 問答快查(濃縮)

Q: 三種演算法差在哪?

A: MiniMax 無剪枝(對照);AlphaBeta 加窗口剪枝;PVS 再加零寬窗口、置換表、靜止搜尋等,實際對局用 PVS。

Q: 時間如何控制?會超時判負嗎?

A: 迭代加深,每層檢查 $\text{total} \times 2 \square \text{movetime}$ 就停;測試程式 kill 後取最後回報手,故非超時即判負,真正問題是根節點排序(已修)。

Q: 為什麼用 Zobrist 而非整個盤面當鍵?

A: Zobrist 可 $O(1)$ 增量更新且分布均勻,適合當雜湊鍵;存整個盤面又慢又佔空間。

Q: 迭代加深不是重複算嗎?會不會浪費?

A: 淺層成本相對很小,且其結果(置換表 + 最佳手排序)讓深層大幅加速,整體反而更快、又能隨時中斷取最佳手。

Q: 評估函數的分數單位?

A: 以兵 $\square 20$ 為基準的內部分;正=走子方有利。將死用 $P_MAX(\pm 100000)$ 表示,遠大於一般評估值。

13. 關鍵參數速查

參數	值 / 條件
P_MAX / M_MAX	+100000 / □100000
置換表	TT_BITS=20 → 約 100 萬筆
靜止搜尋上限	MAX_QDEPTH = 8
材料(評估)	兵20 馬60 象65 車100 后200
LMR	depth□3 且 第3手後 且 非吃子非將軍 → 減1(depth□6 減2)
空著裁剪	depth□3、beta<P_MAX□100、非兵子力□2;R=3(depth□6 為4)
將軍延伸	ply<20 且該手使對方被將軍 → +1 層
期望窗口	±50,失敗 ×3 放寬
兵形罰分	疊兵 □15/個、孤兵 □10